JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ExecutorCompletionService<V>

java.lang.Object
    java.util.concurrent.ExecutorCompletionService<V>

All Implemented Interfaces:

CompletionService<V>

```
public class ExecutorCompletionService<V>
extends Object
implements CompletionService<V>
```

A CompletionService that uses a supplied Executor to execute tasks. This class arranges that submitted tasks are, upon completion, placed on a queue accessible using take. The class is lightweight enough to be suitable for transient use when processing groups of tasks.

Usage Examples. Suppose you have a set of solvers for a certain problem, each returning a value of some type Result, and would like to run them concurrently, processing the results of each of them that return a non-null value, in some method use(Result r). You could write this as:

```
 void solve(Executor e,
            Collection<Callable<Result>> solvers)
     throws InterruptedException, ExecutionException {
     CompletionService<Result> ecs
         = new ExecutorCompletionService<Result>(e);
     for (Callable<Result> s : solvers)
         ecs.submit(s);
     int n = solvers.size();
     for (int i = 0; i < n; ++i) {
         Result r = ecs.take().get();
         if (r != null)
```

```
            use(r);
    }
}
```

Suppose instead that you would like to use the first non-null result of the set of tasks, ignoring any that encounter exceptions, and cancelling all other tasks when the first one is ready:

```
void solve(Executor e,
           Collection<Callable<Result>> solvers)
    throws InterruptedException {
    CompletionService<Result> ecs
        = new ExecutorCompletionService<Result>(e);
    int n = solvers.size();
    List<Future<Result>> futures
        = new ArrayList<Future<Result>>(n);
    Result result = null;
    try {
        for (Callable<Result> s : solvers)
            futures.add(ecs.submit(s));
        for (int i = 0; i < n; ++i) {
            try {
                Result r = ecs.take().get();
                if (r != null) {
                    result = r;
                    break;
                }
            } catch (ExecutionException ignore) {}
        }
    }
    finally {
        for (Future<Result> f : futures)
            f.cancel(true);
    }

    if (result != null)
        use(result);
}
```

Constructor Summary

Constructors

Constructor
`ExecutorCompletionService(Executor executor)`

```
ExecutorCompletionService(Executor executor, BlockingQueue<Future<V>> completionQ
```

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`Future<V>`	`poll()`	Retrieves and removes the Future representing the next completed task, or `null` if none are present.
`Future<V>`	`poll(long timeout, TimeUnit unit)`	Retrieves and removes the Future representing the next completed task, waiting if necessary up to the specified wait time if none are yet present.
`Future<V>`	`submit(Callable<V> task)`	Submits a value-returning task for execution and returns a Future representing the pending results of the task.
`Future<V>`	`submit(Runnable task, V result)`	Submits a Runnable task for execution and returns a Future representing that task.
`Future<V>`	`take()`	Retrieves and removes the Future representing the next completed task, waiting if none are yet present.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ExecutorCompletionService

```
public ExecutorCompletionService(Executor executor)
```

Creates an ExecutorCompletionService using the supplied executor for base task execution and a LinkedBlockingQueue as a completion queue.

Parameters:

executor - the executor to use

Throws:

NullPointerException - if executor is null

ExecutorCompletionService

```
public ExecutorCompletionService(Executor executor,
                                 BlockingQueue<Future<V>> completionQueue)
```

Creates an ExecutorCompletionService using the supplied executor for base task execution and the supplied queue as its completion queue.

Parameters:

executor - the executor to use

completionQueue - the queue to use as the completion queue normally one dedicated for use by this service. This queue is treated as unbounded -- failed attempted Queue.add operations for completed tasks cause them not to be retrievable.

Throws:

NullPointerException - if executor or completionQueue are null

Method Detail

submit

```
public Future<V> submit(Callable<V> task)
```

Description copied from interface: CompletionService

Submits a value-returning task for execution and returns a Future representing the pending results of the task. Upon completion, this task may be taken or polled.

Specified by:

`submit` in interface `CompletionService<V>`

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

submit

```
public Future<V> submit(Runnable task,
                        V result)
```

Description copied from interface: CompletionService

Submits a Runnable task for execution and returns a Future representing that task. Upon completion, this task may be taken or polled.

Specified by:

`submit` in interface `CompletionService<V>`

Parameters:

`task` - the task to submit

`result` - the result to return upon successful completion

Returns:

a Future representing pending completion of the task, and whose get() method will return the given result value upon completion

take

```
public Future<V> take()
              throws InterruptedException
```

Description copied from interface: CompletionService

Retrieves and removes the Future representing the next completed task, waiting if none are yet present.

Specified by:

`take` in interface `CompletionService<V>`

Returns:

the Future representing the next completed task

Throws:

`InterruptedException` - if interrupted while waiting

poll

```
public Future<V> poll()
```

Description copied from interface: `CompletionService`

Retrieves and removes the Future representing the next completed task, or `null` if none are present.

Specified by:

`poll` in interface `CompletionService<V>`

Returns:

the Future representing the next completed task, or null if none are present

poll

```
public Future<V> poll(long timeout,
                      TimeUnit unit)
               throws InterruptedException
```

Description copied from interface: `CompletionService`

Retrieves and removes the Future representing the next completed task, waiting if necessary up to the specified wait time if none are yet present.

Specified by:

`poll` in interface `CompletionService<V>`

Parameters:

`timeout` - how long to wait before giving up, in units of unit

`unit` - a TimeUnit determining how to interpret the timeout parameter

Returns:

the Future representing the next completed task or null if the specified waiting time elapses before one is present

Throws:

```
InterruptedException - if interrupted while waiting
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.